MEMORIA TÉCNICA

Suma de Cuadrados: Programación Secuencial vs. Multiproceso

Nombre y apellidos: Miguel Pérez Matía

1. Entorno de Desarrollo y Configuración del Sistema

Para garantizar la reproducibilidad de los resultados y validar la ejecución bajo el intérprete de Python sin interferencias del sistema anfitrión, se ha configurado un entorno virtual de trabajo dedicado. Las especificaciones del sistema de pruebas se resumen a continuación:

Parámetro de Entorno	Valor de Configuración
Sistema Operativo	`Microsoft Windows 11 Home 64 bits (Versión 10.0.26200)`
Intérprete de Python	`Python 3.14.8 (Entorno Virtual .venv)`
Entorno de Desarrollo	`Visual Studio Code (Terminal PowerShell)`
Nombre del Ejecutable	`suma_cuadrados_secuencial_vs_procesos.py`

Para ejecutar el programa se ejecuta el siguiente comando en la terminal de VS Code:

```
python .\suma_cuadrados_secuencial_vs_procesos.py
```

2. Desarrollo del Trabajo: Caso Principal (Números del 1 al 50)

El algoritmo principal genera una secuencia de 50 números enteros (1 al 50) y los fracciona en lotes de tamaño 7. Para la versión multiproceso, se configura un **`ProcessPoolExecutor`** con **`max_workers = 2`** bajo el contexto de inicio **`spawn`**. A continuación, se reflejan los identificadores de proceso (`PIDs`), lotes y sumas parciales registrados en la consola:

```
PID del proceso principal = 13212

MÉTODO SECUENCIAL
La suma de los cuadrados de los números es 42925

MÉTODO CON PROCESOS
La suma de los cuadrados de los números es 42925

ID lote = 1, PID Worker = 12476, Cantidad de números = 7, Suma parcial = 140

ID lote = 2, PID Worker = 12476, Cantidad de números = 7, Suma parcial = 875

ID lote = 3, PID Worker = 12476, Cantidad de números = 7, Suma parcial = 2296

ID lote = 4, PID Worker = 12476, Cantidad de números = 7, Suma parcial = 4403

ID lote = 5, PID Worker = 12476, Cantidad de números = 7, Suma parcial = 7196

ID lote = 6, PID Worker = 12476, Cantidad de números = 7, Suma parcial = 10675

ID lote = 7, PID Worker = 12476, Cantidad de números = 7, Suma parcial = 14840

ID lote = 8, PID Worker = 12476, Cantidad de números = 1, Suma parcial = 2500

COMPROBACIONES

Se ha calculado correctamente el total de las sumas de los cuadrados de los números (42925) en ambas versiones
Se han procesado correctamente los 50 números en ambas versiones
Se han recibido 8 lotes únicos en la versión con procesos
```

3. Desarrollo del Trabajo: Pruebas Adicionales de Robustez

Para dar cumplimiento a las pruebas límite exigidas en la práctica, se implementó una función de validación que ejecuta tres escenarios de prueba adicionales comprobando el comportamiento ante datos incompletos, vacíos o erróneos:

```
PRUEBAS ADICIONALES

Prueba llamada Lote incompleto con datos [1, 2, 3] y con tamaño 2 de cada lote
Se ha calculado correctamente el total de las sumas de los cuadrados de los números (14) en ambas versiones

Prueba llamada Lista vacía con datos [] y con tamaño 7 de cada lote
Se ha calculado correctamente el total de las sumas de los cuadrados de los números (0) en ambas versiones

Prueba llamada Error en una tarea con datos [1, 'error', 3] y con tamaño 2 de cada lote
Tipo de error: TypeError
No se puede realizar la suma de cuadrados porque la lista contiene uno o más elementos no numéricos
```

4. Desarrollo del Trabajo: Medición y Comparativa de Tiempos

Se ejecutaron 3 mediciones consecutivas del caso principal (1 al 50) registrando los tiempos en segundos con la función **`time.perf_counter()`**. En la versión multiproceso se incluyó todo el ciclo de vida del `pool` (creación y cierre), excluyendo la impresión en pantalla:

```
MEDICIÓN Y COMPARATIVA DE TIEMPOS

EJECUCIÓN          | SECUENCIAL (s)     | MULTIPROCESO (s)
-----------------------------------------------------------
Ejecución 1        | 0.000009           | 0.102781
Ejecución 2        | 0.000010           | 0.100367
Ejecución 3        | 0.000009           | 0.096928
-----------------------------------------------------------
TIEMPO MEDIO       | 0.000009           | 0.100026
-----------------------------------------------------------
La versión secuencial ha sido más rápida que la versión multiproceso
```

Observación del Rendimiento Obtenido: La versión secuencial resulta sensiblemente más rápida (~0.000009 s frente a ~0.100026 s en multiproceso). Este resultado es plenamente coherente con la teoría de sistemas concurrentes: el volumen de cálculo (sumar 50 números) es infinitesimal, por lo que el coste fijo de creación de procesos, carga del intérprete en memoria e intercambio `IPC` domina sobre el tiempo de ejecución puro.

5. Respuestas a las Preguntas Teóricas

a) ¿Cuántos datos, lotes y trabajadores intervienen? ¿Qué has observado en los PIDs?

En la ejecución intervienen **50 datos** *(del 1 al 50) distribuidos en* **8 lotes***, gestionados por un* ***pool*** *configurado para un* ***máximo de 2 trabajadores*** *(`max_workers = 2`).*

Sin embargo, en la terminal (imagen en la página 1) se observa que ***un único trabajador*** *(`PID = 12476`) procesó todos los lotes consecutivamente. Esto se debe a que el `pool` **reutiliza** los procesos para evitar el alto coste computacional* **(overhead***) de crearlos y destruirlos por cada tarea. Como el cálculo de estos lotes es tan rápido, este primer trabajador logra vaciar toda la cola antes de que el sistema tenga la necesidad de instanciar un segundo proceso.*

b) ¿Por qué devuelve cada trabajador su resultado en lugar de acumularlo en una variable global del principal?

*Debido a que cada proceso posee **un** **espacio de memoria virtual totalmente aislado** por el sistema operativo. Si un trabajador modificara una variable global, el cambio solo afectaría a su propia memoria local y el proceso principal jamás lo percibiría.*

Por ello, para transmitir información entre procesos independientes, los trabajadores deben devolver sus resultados mediante mecanismos de comunicación **IPC** *(**Inter-Process Communication**), serializando la información (con librerías como `pickle`) y transmitiéndola de vuelta al proceso principal a través de tuberías o colas de memoria*

c) ¿Para qué sirven spawn, la protección __main__ y el bloque with?

En Windows, el método de inicio de procesos es **spawn***, el cual **arranca un intérprete** de Python totalmente limpio e independiente. Esto garantiza un aislamiento completo de la memoria, pero exige al proceso hijo volver a importar el script original.*

Por este motivo es imprescindible la protección **`if __name__ == '__main__'`***, ya que evita que la reimportación del módulo ejecute de nuevo el código de creación de procesos, lo que generaría un bucle recursivo infinito que colapsaría el sistema.*

Finalmente, envolver la ejecución en un bloque **`with`** *actúa como gestor de contexto: frena el proceso principal hasta que concluyen las tareas y libera automáticamente los recursos del `pool`, evitando la existencia de procesos huérfanos o zombis.*

d) ¿Cómo recoges los resultados? Si usas map, ¿su orden de entrega demuestra el orden en que terminaron las tareas?

*Los resultados se recogen en el proceso principal mediante la instrucción **`list(pool.map(calcular_suma_cuadrados, lotes))`**, la cual devuelve una lista con las sumas parciales calculadas por los trabajadores.*

El orden de la lista devuelta no demuestra el orden en que terminaron físicamente las tareas en la CPU*. La función **`pool.map()`** garantiza siempre **la entrega en el*** **mismo orden** en que se **pasaron los elementos** en el iterable de entrada*; por lo tanto, aunque un trabajador finalice un lote posterior antes que otro, el sistema retiene los resultados internamente para reconstruir y devolver la secuencia original exacta.*

6. Conclusión

Esta práctica ha permitido entender de forma práctica la gestión de procesos en Python. Hemos aprendido a dividir cargas de trabajo en lotes, coordinar su ejecución mediante un pool *y comprobar el comportamiento real del sistema operativo respecto al aislamiento de memoria y la reutilización de procesos. Asimismo, hemos experimentado empíricamente que el multiprocesamiento solo aporta ventajas en tareas computacionalmente pesadas y hemos validado el control de errores en entornos concurrentes.*

Declaración de Apoyo e Inteligencia Artificial: *Se ha utilizado asistencia de IA (Gemini Notebook) como apoyo para la revisión de sintaxis del código Python y la maquetación estructurada del presente documento en formato Word.*